A Full-Stack Autonomous Quadrotor System for Safe Navigation in Unknown Environments

1st Armon Barakchi

I. Introduction

Safe autonomous navigation in unknown, obstacle-rich environments is a fundamental challenge in robotics. A robot operating in such an environment must simultaneously estimate its own state from noisy sensors, build a local map of its surroundings from onboard perception, plan a collision-free path to a goal, and guarantee that its motion remains safe even in the presence of estimation error and planning approximations. While each of these subproblems has been studied extensively in isolation, integrating them into a cohesive, real-time system that provides formal safety guarantees remains an open and practically important problem.

Existing approaches typically treat planning and safety as a single problem, embedding obstacle avoidance directly into a trajectory optimizer [1], [2]. While effective when the full obstacle map is known in advance, these methods are difficult to extend to unknown environments where obstacles are discovered online through local sensing. An alternative line of work uses Control Barrier Functions (CBFs) [3] to enforce safety as a real-time constraint on the control input, independent of the planner. CBFs provide formal guarantees that the system remains in a safe set for all time, but they suffer from local minima when used alone for navigation: a drone commanded toward a goal directly behind an obstacle will be pushed away indefinitely with no mechanism to navigate around it [3]. This motivates a layered architecture in which a planner handles global navigation while a CBF handles local safety.

In this work, we present a full-stack autonomous quadrotor simulation system, modeling a *crazyflie 2.1 drone*, that integrates four components into a cohesive pipeline for safe waypoint navigation in unknown environments:

1) A **Multiplicative Unscented Kalman Filter (MUKF)** on $SE(3)$ that fuses noisy IMU measurements at 500 Hz and GPS at 10 Hz to estimate the drone's position, velocity, attitude, and sensor biases [4], [5].
2) A **simulated spinning LiDAR** that casts 648 rays per scan using MuJoCo's raycasting engine, clusters hit points into sphere estimates, and maintains a local obstacle map with conservative radius inflation to absorb center estimation error [6].
3) A **Real-Time Adaptive A* (RTAA*) motion planner** [7] that replans at 10 Hz on a discretized 3D grid using only the current LiDAR map. RTAA* is specifically chosen for its real-time guarantee and natural compatibility with local sensing: it requires no global map and improves its heuristic estimates online as the drone explores.
4) A **Control Barrier Function (CBF) safety filter** [3], [8] that runs as a quadratic program (QP) at every control step, modifying the planner's output force vector to guarantee the drone remains outside all detected obstacle safety radii, regardless of planner quality or state estimation error.

These components are layered so that their responsibilities are cleanly separated: the MUKF provides state estimates, LiDAR provides the obstacle map, RTAA* handles global navigation, and the CBF provides the safety guarantee. A geometric controller on $SE(3)$ [9] translates RTAA* waypoints into force and torque commands, which the CBF then filters before application.

We evaluate the system in MuJoCo [10] across four qualitatively distinct maps — a corridor with a narrow gap with two goal waypoints , a dense column forest with three goal waypoints, a sequence of gate frames with three goal waypoints, and a rising spiral of obstacles with three goal waypoints — and compare performance across ablations that isolate the contribution of each component.

The remainder of the paper is organized as follows. Section II formalizes the quadrotor dynamics, state estimation problem, and safety constraint. Section III describes each component of the pipeline in detail. Section IV presents experimental results. Section V concludes with limitations and directions for future work.

II. Problem Formulation

A. Quadrotor Dynamics on SE(3)

We model the quadrotor as a rigid body whose configuration lies on the Special Euclidean group $SE(3) = \mathbb{R}^3 \times SO(3)$. The state is $x = (p, v, R, \omega)$ where $p \in \mathbb{R}^3$ is the position in the world frame, $v \in \mathbb{R}^3$ is the linear velocity in the world frame, $R \in SO(3)$ is the rotation matrix from body to world frame, and $\omega \in \mathbb{R}^3$ is the angular velocity in the body frame. The continuous-time equations of motion are [9]:

$$\dot{p} = v \tag{1}$$

$$\dot{v} = \frac{F}{m} R e_3 - g e_3 \tag{2}$$

$$\dot{R} = R\hat{\omega} \tag{3}$$

$$J\dot{\omega} = M - \omega \times J\omega \tag{4}$$

where $m \in \mathbb{R}_{>0}$ is the drone mass, g is gravitational acceleration, $e_3 = [0,0,1]^\top$, $F \in \mathbb{R}$ is the total collective thrust, $M \in \mathbb{R}^3$ is the torque vector, $J \in \mathbb{R}^{3\times3}$ is the inertia tensor, and $\hat{\omega}$ denotes the skew-symmetric matrix satisfying $\hat{\omega}v = \omega \times v$ for all $v \in \mathbb{R}^3$. The control inputs are the thrust-torque pair (F, M), subject to the physical constraint $0 \le F \le F_{\max}$.

B. State Estimation Problem

The drone's true state x is not directly observable. Instead, at each timestep t the drone receives two noisy measurements:

IMU (500 Hz): An accelerometer and gyroscope mounted at the center of mass provide body-frame specific force and angular velocity:

$$a_{\text{meas}} = R^\top(\dot{v} + ge_3) + b_a + \eta_a \tag{5}$$

$$\omega_{\text{meas}} = \omega + b_g + \eta_g \tag{6}$$

where $b_a \in \mathbb{R}^3$ and $b_g \in \mathbb{R}^3$ are slowly-varying accelerometer and gyroscope biases, and $\eta_a \sim \mathcal{N}(0, \sigma_a^2 I)$, $\eta_g \sim \mathcal{N}(0, \sigma_g^2 I)$ are white noise processes with $\sigma_a = 0.02\,\text{m/s}^2$ and $\sigma_g = 10^{-3}$ rad/s.

GPS (10 Hz): A noisy position measurement in the world frame:

$$p_{\text{meas}} = p + \eta_{\text{GPS}}, \quad \eta_{\text{GPS}} \sim \mathcal{N}(0, \sigma_{\text{GPS}}^2 I) \tag{7}$$

with $\sigma_{\text{GPS}} = 0.3\,\text{m}$.

The state estimation problem is to maintain a posterior estimate $\hat{x} = (\hat{p}, \hat{v}, \hat{R}, \hat{\omega})$ of the full state along with bias estimates $(\hat{b}_a, \hat{b}_g)$, given the history of IMU and GPS measurements. The extended state vector including biases has dimension 16 (position 3, velocity 3, quaternion 4, gyro bias 3, accel bias 3). Because the attitude component lives on $SO(3)$ rather than $\mathbb{R}^n$, standard additive Kalman filters are not directly applicable; we address this in Section III-A.

C. Obstacle Model and Perception

The environment contains N static spherical obstacles with centers $c_i \in \mathbb{R}^3$ and visual radii $r_i^{\text{vis}} > 0$, $i = 1, \ldots, N$. The obstacle positions are *unknown a priori* and must be discovered through onboard sensing. At each scan, the drone's LiDAR sensor returns a set of hit points $\mathcal{H}_t \subset \mathbb{R}^3$ from which obstacle centers $\hat{c}_i$ and estimated radii $\hat{r}_i$ must be inferred. Due to the geometry of LiDAR raycasting — rays strike only the near face of each sphere — the raw hit point centroid underestimates the true center. We account for this estimation error through a conservative radius inflation:

$$r_i^{\text{safe}} = \hat{r}_i + \delta \tag{8}$$

where $\delta > 0$ is a safety margin chosen to absorb the expected center estimation error. The *local obstacle map* at time t is the set $\mathcal{M}_t = \{(\hat{c}_i, r_i^{\text{safe}})\}$ of currently detected obstacles, which is updated at each LiDAR scan.

D. Safety Constraint via Control Barrier Functions

For each detected obstacle i, we define the barrier function:

$$h_i(p) = \|p - \hat{c}_i\|^2 - (r_i^{\text{safe}})^2 \tag{9}$$

The *safe set* is $\mathcal{C} = \{p \in \mathbb{R}^3 : h_i(p) \ge 0,\ \forall i\}$, i.e. the region outside all safety radii. A continuously differentiable function h_i is a valid Control Barrier Function if there exists a class-$\mathcal{K}$ function α such that [3]:

$$\dot{h}_i(x) \ge -\alpha(h_i(p)) \tag{10}$$

for all $x \in \mathcal{C}$. Under this condition, $\mathcal{C}$ is forward-invariant: if the drone starts in $\mathcal{C}$, it remains in $\mathcal{C}$ for all future time. Taking $\alpha(h) = \alpha h$ for $\alpha > 0$ and expanding $\dot{h}_i$ along the system trajectories:

$$\dot{h}_i = \underbrace{2(p - \hat{c}_i)^\top v}_{\dot{h}_{\text{vel}}} + \underbrace{\frac{2}{m}(p - \hat{c}_i)^\top F}_{\dot{h}_{\text{ctrl}}} \ge -\alpha h_i(p) \tag{11}$$

where the first term $\dot{h}_{\text{vel}}$ depends only on the current velocity and is not directly controllable, while the second term $\dot{h}_{\text{ctrl}}$ is affine in the control force F and can be shaped by the controller. Rearranging (11) yields a linear constraint on F that is enforced at each timestep via a quadratic program, as described in Section III-D.

E. Navigation Problem

The navigation task is defined by a sequence of K goal positions $\{g_1, \ldots, g_K\} \subset \mathbb{R}^3$. Because obstacle positions are unknown at planning time and discovered online, a stated goal g_k may fall inside the safety radius of a detected obstacle. Flying directly to such a goal would create a conflict between the goal-seeking controller and the CBF safety filter, resulting in oscillation or deadlock. We therefore define the *safe goal* g_k^s as the projection of g_k onto the free space $\mathcal{C}$:

$$g_k^s = \arg\min_{z \in \mathcal{C}} \|z - g_k\| \tag{12}$$

For spherical obstacles, this projection has a closed form. If g_k lies inside the safety radius of obstacle i (i.e. $\|g_k - \hat{c}_i\| < r_i^{\text{safe}} + \varepsilon$ for a small margin $\varepsilon > 0$), then g_k^s is the point on the boundary of the safety radius closest to g_k:

$$g_k^s = \hat{c}_i + \frac{g_k - \hat{c}_i}{\|g_k - \hat{c}_i\|} \cdot (r_i^{\text{safe}} + \varepsilon) \tag{13}$$

When multiple obstacles affect g_k, the projection is applied sequentially. When $g_k \in \mathcal{C}$ already, $g_k^s = g_k$ and the problem reduces to standard waypoint navigation.

The full navigation problem is then: given only onboard IMU, GPS, and LiDAR measurements, find a control policy π that minimizes the distance to the sequence of safe goals while maintaining the safety constraint at all times:

$$\begin{aligned} \underset{\pi}{\text{minimize}} \quad & \sum_{k=1}^{K} \|p(t_k) - g_k^s\| \\ \text{subject to} \quad & h_i(p(t)) \geq 0 \quad \forall i \in \mathcal{M}_t,\ \forall t \geq 0 \\ & \dot{x} = f(x, \pi(x)), \quad 0 \leq F \leq F_{\max} \\ & \mathcal{M}_t \text{ updated online from LiDAR} \end{aligned} \tag{14}$$

where t_k is the time at which goal k is reached, and g_k^s depends on $\mathcal{M}_{t_k}$ — the obstacle map at the time of arrival. This formulation captures the adaptive nature of the problem: as new obstacles are detected, the effective targets g_k^s may shift, and the planner must respond accordingly.

III. Technical Approach

The quadrotor pipeline runs at 500 Hz and is organized into four subsystems that execute at different rates, as illustrated in Figure 1. At every timestep, the UKF predicts the drone's state from IMU measurements. Every 50 steps (≈10 Hz), GPS updates the UKF, LiDAR scans the environment and updates the obstacle map, and RTAA* replans an intermediate waypoint toward the current safe goal. At every timestep, the geometric controller computes a nominal force from the current waypoint, and the CBF filters it before application to the simulator.

500 Hz:	IMU → UKF predict → Geometric ctrl → CBF filter → MuJoCo
10 Hz:	GPS → UKF update
10 Hz:	LiDAR scan → Obstacle map update
10 Hz:	RTAA* replan → Intermediate waypoint

Fig. 1: SE3-Quad pipeline. UKF runs at full IMU rate; perception, planning, and GPS correction run at 10 Hz.

A. State Estimation: UKF with Multiplicative Attitude Error State

Standard Kalman filtering assumes the state lives in a vector space so that perturbations can be added directly. This assumption fails for the attitude component $R \in SO(3)$: adding an arbitrary perturbation δR to a rotation matrix does not in general produce another rotation matrix. We address this with a Multiplicative UKF (MUKF) [4] that maintains the mean attitude as a unit quaternion $q \in S^3$ and represents perturbations in the Lie algebra $\mathfrak{so}(3) \cong \mathbb{R}^3$ via the exponential map.

1) Error State Parameterization: The full state is parameterized as $x = (p, v, q, b_g, b_a)$ with q the unit quaternion representing R. The *error state* $\delta x \in \mathbb{R}^{15}$ is:

$$\delta x = \begin{bmatrix} \delta p \\ \delta v \\ \delta \theta \\ \delta b_g \\ \delta b_a \end{bmatrix} \in \mathbb{R}^{15} \tag{15}$$

where $\delta p, \delta v \in \mathbb{R}^3$ are additive position and velocity errors, $\delta\theta \in \mathbb{R}^3$ is the attitude error in rotation vector form (Lie algebra), and $\delta b_g, \delta b_a \in \mathbb{R}^3$ are bias errors. A perturbed state is composed from the nominal mean as:

$$q_i = q \otimes \exp\left(\tfrac{1}{2}\delta\theta_i\right) \tag{16}$$

where $\otimes$ denotes quaternion multiplication and $\exp(\cdot)$ is the quaternion exponential map. This ensures all sigma point attitudes remain on S^3 – the set of all unit quaternions.

2) Sigma Point Propagation: Given mean state (p, v, q, b_g, b_a) and error-state covariance $P \in \mathbb{R}^{15\times15}$, we generate $2n + 1 = 31$ sigma points by perturbing the error state along the columns of $\sqrt{(n+\lambda)P}$ (Cholesky factor), with $\lambda = \alpha^2(n + \kappa) - n$, $\alpha = 10^{-2}$, $\beta = 2$, $\kappa = 0$. Each sigma point is propagated through the nonlinear IMU dynamics:

$$p_i^+ = p_i + v_i \Delta t \tag{17}$$

$$v_i^+ = v_i + (R_i(a_{\text{meas}} - b_{a,i}) - g e_3)\,\Delta t \tag{18}$$

$$q_i^+ = q_i \otimes \exp\left(\tfrac{1}{2}(\omega_{\text{meas}} - b_{g,i})\Delta t\right) \tag{19}$$

where R_i is the rotation matrix corresponding to q_i.

3) Mean Recovery and Covariance Update: The predicted mean position and velocity are recovered by the standard weighted sum. For the predicted mean attitude we use the geodesic mean on $SO(3)$: iterate

$$q^+ \leftarrow q^+ \otimes \exp\left(\sum_i W_i^m \log({q^+}^{-1} \otimes q_i^+)\right) \tag{20}$$

until convergence, where $\log(\cdot)$ is the quaternion logarithm. The predicted error-state covariance is:

$$P^- = \sum_i W_i^c\, \delta x_i\, \delta x_i^\top + Q\Delta t \tag{21}$$

where δx_i is the error between sigma point i and the predicted mean computed in the error-state space (using $\log$ for the attitude component), and Q is the process noise covariance.

4) GPS Measurement Update: The GPS measurement function is $h(x) = p$, which is linear in the error state ($H = [I_{3\times3}\ 0_{3\times12}]$). The standard Kalman update applies:

$$S = HP^-H^\top + R_{\text{GPS}} \tag{22}$$

$$K = P^-H^\top S^{-1} \tag{23}$$

$$\delta x = K(p_{\text{meas}} - \hat{p}^-) \tag{24}$$

The state is then corrected additively for position, velocity, and biases, and multiplicatively for attitude:

$$q \leftarrow \hat{q}^- \otimes \exp\left(\tfrac{1}{2}\delta\theta\right) \tag{25}$$

The covariance is updated using the Joseph form for numerical stability: $P = (I - KH)P^-(I - KH)^\top + KR_{\text{GPS}}K^\top$.

B. Perception: LiDAR Obstacle Detection

The simulated LiDAR sensor casts $N_h \times N_v = 72 \times 9 = 648$ rays per scan in a full 360° horizontal by 90° vertical field of view, with a maximum range of $d_{\max} = 2.0\,\text{m}$. Each ray r_j is defined in the body frame and rotated to the world frame by $\hat{R}$. Raycasting is performed via MuJoCo's `mj_ray()` function [10] with two exclusions that prevent spurious detections:

1) **Body exclusion**: the drone's own geometry is excluded by passing the drone body ID as the `bodyexclude` argument.
2) **Geom group masking**: obstacles are assigned to geom group 1 and the ground plane and drone geoms to group 0. A group mask $[0, 1, 0, 0, 0, 0]$ ensures only obstacle spheres are raycasted.

1) Hit Point Clustering: Raw hit points $\mathcal{H} = \{p + d_j r_j : d_j < d_{\max}\}$ are clustered into obstacles using a DBSCAN-style procedure with radius $\varepsilon_c = 0.35\,\text{m}$ and minimum cluster size $n_{\min} = 3$. For each cluster $\mathcal{H}_k$:

Center estimation: Hit points lie on the near face of the obstacle sphere, so the raw centroid $\bar{h}_k = \frac{1}{|\mathcal{H}_k|} \sum_{h \in \mathcal{H}_k} h$ underestimates the true center. We correct for this by pushing the centroid away from the drone along the line of sight:

$$\hat{c}_k = \bar{h}_k + \frac{\bar{h}_k - p}{\|\bar{h}_k - p\|} \cdot \hat{r}_k \tag{26}$$

where $\hat{r}_k = \max\left(\max_{h \in \mathcal{H}_k} \|h - \bar{h}_k\|,\ 0.08\right)$ is the estimated sphere radius from the spread of hit points.

Safe radius: The safe radius passed to the CBF and planner inflates the estimated radius by a fixed margin $\delta = 0.20\,\text{m}$ to absorb center estimation error:

$$r_k^{\text{safe}} = \hat{r}_k + \delta \tag{27}$$

C. Motion Planning: Real-Time Adaptive A*

Global navigation is handled by Real-Time Adaptive A* (RTAA*) [7], an online replanning algorithm that is naturally compatible with local sensing: it operates on whatever obstacle information is currently available and does not require a global map.

1) Grid Discretization: The workspace $[-1.5, 1.5]^2 \times [0.3, 2.3]\,\text{m}$ is discretized into a 6-connected grid with resolution $\Delta = 0.20\,\text{m}$, giving approximately $15 \times 15 \times 10 = 2{,}250$ cells. A cell is marked *occupied* if any detected obstacle center lies within the planning radius r_{plan} of the cell center (in grid coordinates, this reduces to an integer distance check). The planning radius $r_{\text{plan}} = 0.27\,\text{m}$ is set slightly smaller than the CBF safety radius to ensure the planned path is feasible for the safety filter.

2) RTAA Algorithm:* At each replanning step, RTAA* runs a limited A^* search from the drone's current grid cell toward the goal, expanding at most $k = 30$ nodes. Let $h(n)$ denote the heuristic at node n (Euclidean distance to goal, overridden by learned values) and $g(n)$ the cost from start. After the search, there are two steps:

1) **Heuristic update (learning step):** The heuristic of the start node is updated to the minimum f-value remaining in the open list:

$$h(s) \leftarrow \max\left(h(s),\ \min_{n \in \text{OPEN}} f(n) - g(s)\right) \tag{28}$$

This prevents the algorithm from revisiting the same node with an overoptimistic estimate, guaranteeing finite convergence to the goal [7].
2) **Waypoint extraction:** The next intermediate waypoint is the world-frame position of the grid cell one step from the start along the best path found.

RTAA* replans every 50 simulation steps (≈10 Hz). At a resolution of $\Delta = 0.20\,\text{m}$ and lookahead $k = 30$, each call takes approximately $0.13\,\text{ms}$ — well within the real-time budget of a 500 Hz control loop. When the goal changes (next goal in the sequence), the learned heuristic table is cleared via `reset_learning()` to prevent stale estimates from interfering with the new search.

3) Safe Goal Projection: Before replanning, the current goal g_k is projected onto the free space as defined in (13). The projected safe goal g_k^s is passed to RTAA* as the search target, ensuring the planner never routes the drone toward a location that would conflict with the CBF safety filter.

D. Safety Filtering: Control Barrier Functions

The CBF safety filter runs at the full 500 Hz control rate and operates as a minimal-intervention safety layer: it modifies the nominal control force only when necessary to prevent constraint violation.

1) QP Formulation: Given the nominal force $F_{\text{nom}} \in \mathbb{R}^3$ from the geometric controller, the CBF filter solves the quadratic program:

$$\begin{aligned} F_{\text{safe}} = \underset{F \in \mathbb{R}^3}{\arg\min} \quad & \|F - F_{\text{nom}}\|^2 \\ \text{subject to} \quad & \frac{2}{m}(p - \hat{c}_i)^\top F \geq -\alpha h_i(p) - \dot{h}_{i,\text{vel}} \quad \forall i \in \mathcal{A} \\ & -F_{\max} \leq F_j \leq F_{\max} \quad j = 1, 2, 3 \end{aligned} \tag{29}$$

where $\dot{h}_{i,\text{vel}} = 2(p - \hat{c}_i)^\top v$ is the velocity contribution to $\dot{h}_i$ (not directly controllable), $\alpha > 0$ is the class-$\mathcal{K}$ gain, and $\mathcal{A}$ is the set of active obstacles. The per-component force bounds $|F_j| \leq F_{\max}$ are used in place of the norm constraint $\|F\| \leq F_{\max}$ because the latter is a second-order cone constraint that the OSQP solver [11] cannot handle; the linear bounds keep the problem a standard QP. The problem is solved using OSQP with warm starting, typically converging in under $0.5\,\text{ms}$.

2) Activation Gating: A key design choice is that the CBF constraint for obstacle i is only included in (29) when two conditions are simultaneously met:

1) The drone is within the activation distance: $\|p - \hat{c}_i\| < r_i^{\text{safe}} + d_{\text{act}}$, with $d_{\text{act}} = 0.5\,\text{m}$.
2) The drone is approaching the obstacle: $\dot{h}_{i,\text{vel}} < 0$.

Without this gating, the CBF constraint is violated by the nominal force whenever F_{nom} has a component pointing toward a distant obstacle (for example, when the goal lies beyond the

obstacle). This causes the CBF to modify F by hundreds of millinewtons even when the drone is safely far away, fighting the planner unnecessarily. With gating, the CBF intervenes only when the drone is both close enough and moving in a direction that would carry it toward the obstacle — the precise condition under which the barrier function is at risk of being violated. When inside the safety radius ($h_i < 0$), the approach condition is waived and the constraint is always enforced regardless of velocity direction.

3) Relationship Between Planning and Safety Radii: The planning radius $r_{\text{plan}} = 0.27\,\text{m}$ used by RTAA* is set smaller than the CBF safety radius $r^{\text{safe}} = 0.30\,\text{m}$. This hierarchy ensures that paths found by RTAA* have sufficient clearance for the CBF to execute them without modification in nominal conditions: the planned path stays outside r^{safe}, so the CBF constraint is never active along the nominal path. The CBF then acts as a true safety backstop, engaging only when disturbances, estimation error, or planning imprecision carry the drone toward an obstacle boundary.

E. Geometric Controller on SE(3)

The geometric controller [9] tracks RTAA* intermediate waypoints by computing force and torque commands directly on $SE(3)$, avoiding the singularities present in Euler-angle formulations.

1) Position Loop: Given the current UKF state estimate $(\hat{p}, \hat{v}, \hat{R}, \hat{\omega})$ and target waypoint p_{des} (from RTAA* or the safe goal), the nominal force is computed as:

$$F_{\text{nom}} = -k_p(\hat{p} - p_{\text{des}}) - k_v\hat{v} + mge_3 \tag{30}$$

with $k_p = 2.0$, $k_v = 1.2$. If $\|F_{\text{nom}}\| > F_{\max}$, it is clipped to $F_{\max}$ before being passed to the CBF filter.

2) Attitude Loop: The desired thrust direction is $\hat{n} = F_{\text{safe}}/\|F_{\text{safe}}\|$. The attitude error is computed directly on $SO(3)$ as:

$$e_R = \frac{1}{2}(R_{\text{des}}^\top \hat{R} - \hat{R}^\top R_{\text{des}})^\vee \tag{31}$$

where $(\cdot)^\vee : \mathfrak{so}(3) \to \mathbb{R}^3$ is the vee map, and $R_{\text{des}} \in SO(3)$ is the desired rotation matrix constructed from $\hat{n}$ and a yaw reference. The torque is:

$$M = -k_R e_R - k_\omega(\hat{\omega} - \omega_{\text{des}}) + \hat{\omega} \times J\hat{\omega} \tag{32}$$

with $k_R = 0.5$, $k_\omega = 0.1$. The torque is expressed in the world frame as $\tau = \hat{R}M$ and applied alongside F_{safe} via MuJoCo's `xfrc_applied` interface.

F. Pipeline Integration

The four subsystems are integrated so that their failure modes do not compound. The UKF is used for state feedback to the position loop and RTAA*, but the CBF filter uses the true simulator position rather than the UKF estimate. This is because UKF position error ($\sim$0.15 m mean) could cause the CBF to underestimate the true distance to an obstacle, potentially allowing a constraint violation. Using the true position for safety filtering is a deliberate architectural choice: on a real platform, this role would be filled by a higher-accuracy localization source (e.g., a motion capture system or fused LiDAR-SLAM estimate) rather than GPS-only UKF.

Similarly, the goal projection in (13) is recomputed at every RTAA* replan step using the current obstacle map $\mathcal{M}_t$. As new obstacles are detected by LiDAR, the effective target g_k^s may shift, and RTAA* automatically routes around the updated obstacle field in its next replan cycle.

IV. Results

We evaluate SE3-Quad in MuJoCo [10] across three experiments designed to isolate the contribution of each pipeline component and validate performance across qualitatively distinct environments. All experiments use a Crazyflie-class quadrotor model with mass $m = 0.035\,\text{kg}$, maximum thrust $F_{\max} = 0.6\,\text{N}$, and simulation timestep $\Delta t = 2\,\text{ms}$. Sensor noise parameters match Section II: $\sigma_{\text{GPS}} = 0.3\,\text{m}$, $\sigma_a = 0.02\,\text{m/s}^2$, $\sigma_g = 10^{-3}\,\text{rad/s}$, with additive biases $b_a = [0.01, 0.02, -0.01]^\top\,\text{m/s}^2$ and $b_g = [0.005, -0.003, 0.002]^\top\,\text{rad/s}$. Unless stated otherwise, all results use the full pipeline with $\alpha = 1.0$, $k = 30$, $r^{\text{safe}} = 0.30\,\text{m}$, $r_{\text{plan}} = 0.27\,\text{m}$, and activation distance $d_{\text{act}} = 0.5\,\text{m}$.

A. Experiment 1: CBF Effectiveness

To isolate the contribution of the CBF safety filter, we run the full pipeline and the pipeline with CBF disabled (`--no_cbf`) on all four maps. With CBF disabled, RTAA* still plans collision-free paths but provides no formal safety guarantee: any planning approximation, LiDAR estimation error, or state estimation offset can carry the drone across an obstacle boundary without correction. Table I reports MuJoCo contact events (physical collisions detected by the simulator between drone and obstacle geoms), CBF safety radius violations (timesteps with $h_i(\hat{p}) < 0$ for any obstacle i), the fraction of timesteps at which at least one CBF constraint was active, and the number of goals successfully reached within the 60 s time limit.

TABLE I: CBF effectiveness across maps. "Contacts" are physical MuJoCo collisions. "Violations" are timesteps inside the CBF safety radius ($h_i < 0$), which is a stricter criterion than physical contact. Full pipeline vs. CBF disabled.

Map	Mode	Contacts	Violations	CBF Active (%)	Goals
Corridor	Full pipeline	0	150	41.3	2
	No CBF	336	3030	—	2
Forest	Full pipeline	0	3332	76.8	3
	No CBF	1307	4981	—	3
Gates	Full pipeline	0	0	42.5	3
	No CBF	0	495	—	3
Spiral	Full pipeline	0	3460	51.4	3
	No CBF	88	3303	—	3

The CBF activation rate reflects how often the safety filter intervenes in each environment. contacts and violations is important: a CBF violation (entering the safety radius) does not necessarily imply a physical collision, since the visual obstacle radius $r^{\text{vis}} = 0.15\,\text{m}$ is smaller than the safety radius $r^{\text{safe}} = 0.30\,\text{m}$. Physical safety is guaranteed as long as contacts remain zero, even if CBF violations occur.

B. Experiment 2: Planner Necessity

To isolate the contribution of RTAA*, we compare the full pipeline against CBF-only navigation (`--no_rtaa`), where the drone flies directly toward each goal with only the CBF filter for obstacle avoidance. Without a planner, the drone is susceptible to local minima: when the goal lies on the far side of an obstacle, the CBF repels the drone while the position controller attracts it toward the goal, producing a deadlock in which the drone makes no forward progress. Table II characterizes this behavior across maps. Note that if Time to Complete is 60 seconds, the goals were not reached in the full simulation time.

TABLE II: Planner necessity. "Stuck" indicates the drone made no progress toward the current goal for more than 5 s, indicating a local minimum. Full pipeline vs. CBF only (RTAA* disabled).

Map	Mode	Contacts	Goals	Stuck	Time to Complete (s)
Corridor	Full pipeline	0	2	No	13.5
	CBF only	0	2	No	6.8
Forest	Full pipeline	0	3	No	20.5
	CBF only	60	2	Yes	60
Gates	Full pipeline	0	3	No	17.5
	CBF only	29	2	Yes	60
Spiral	Full pipeline	0	3	No	31.5
	CBF only	116	3	No	42.5

This experiment illustrates the complementary roles of RTAA* and the CBF. The CBF alone cannot guarantee goal reachability because it provides no mechanism to route around obstacles [3]. RTAA* alone cannot guarantee safety because its grid resolution and LiDAR estimation error may produce paths that clip obstacle boundaries. Together, RTAA* eliminates local minima by finding intermediate waypoints that circumnavigate obstacles, while the CBF enforces the safety constraint at every timestep regardless of planning imprecision.

C. Experiment 3: Map Validation

We evaluate the full pipeline on four maps of increasing navigational complexity. Each map is designed to stress a different aspect of the system:

- **Corridor**: Two parallel walls of obstacles with a navigable gap. Tests whether RTAA* can discover and route through a gap that is initially unknown, using only the local LiDAR map built online.
- **Forest**: Six asymmetrically placed pillars at varying heights. Tests multi-obstacle avoidance across three goals at different altitudes, requiring the planner to thread a path through a cluttered 3D environment.
- **Gates**: Two gate frames constructed from pairs of obstacle spheres at different heights. Tests precision navigation through narrow openings where the CBF must deflect the drone away from gate posts without preventing forward passage.
- **Spiral**: Five obstacles arranged in a rising spiral pattern. Tests 3D path planning that requires simultaneous lateral navigation and altitude gain, exercising the full 3D capability of the RTAA* planner.

Table III reports full pipeline performance on each map. "Min obs dist" is the minimum distance from the true drone position to any obstacle surface recorded during the run; negative values indicate the drone entered the CBF safety radius (a CBF violation) but not necessarily the physical obstacle. "CBF active" is the fraction of timesteps at which at least one CBF constraint was enforced. "UKF error" is the mean position estimation error $\|\hat{p} - p_{\text{true}}\|$ over the run.

TABLE III: Full pipeline performance across all four validation maps. All runs use the full UKF + RTAA* + CBF pipeline. "Min obs dist" is distance to nearest obstacle surface; negative indicates a CBF safety radius violation (not physical contact). Goals format is reached/total.

Map	Goals	Contacts	Min Obs Dist (m)	CBF Active (%)	UKF Error (m)
Corridor	2	0	0.127	41.3	0.176
Forest	3	0	0.136	76.8	0.157
Gates	3	0	0.169	42.5	0.190
Spiral	3	0	0.05	51.4	0.151

D. Discussion

The results across Experiments 1–3 demonstrate that all three active components of the pipeline are necessary for reliable safe navigation. Removing the CBF (Experiment 1) results in physical collisions when planning or estimation errors carry the drone across an obstacle boundary. Removing RTAA* (Experiment 2) results in local minima where the drone deadlocks between the goal-seeking controller and CBF repulsion, failing to reach goals that lie behind obstacles.

A key design decision validated by these results is the relationship between the planning radius and the CBF safety radius. By setting $r_{\text{plan}} < r^{\text{safe}}$, RTAA* plans paths with clearance for the CBF to execute safely: the nominal path already satisfies the safety constraint, so the CBF modifies the force only when disturbances or estimation error carry the drone off the planned path. When this hierarchy is violated, the CBF begins fighting the planner, producing erratic behavior. The 0.03 m gap used here ($r_{\text{plan}} = 0.27\,\text{m}$, $r^{\text{safe}} = 0.30\,\text{m}$) was empirically found to provide a good balance.

The activation gating condition (Section III-D) is also validated by these results. Without gating, the CBF constraint fires whenever the nominal force points in the general direction of a distant obstacle, producing force modifications of 400–600 mN even at distances greater than 1 m. With gating, the CBF is quiescent when the drone is safely far from obstacles and intervenes sharply only when the drone is both within d_{act} of an obstacle and approaching it. This behavior is consistent with the CBF activation rates in Table III: the filter is inactive for the majority of each run and engages selectively near obstacle boundaries.

Finally, the safe goal projection (13) is essential when goals lie close to obstacles. Without it, the position controller and CBF enter a persistent conflict — the controller drives toward the goal while the CBF repels the drone from the nearby obstacle — that is resolved only by projecting the effective target to the nearest feasible point outside the safety radius. This modification has no effect when goals are in open space

and is activated automatically as the online obstacle map is built.

V. Conclusion

We presented a full-stack autonomous quadrotor navigation system that integrates four components — a Multiplicative UKF for state estimation, a simulated LiDAR for online obstacle detection, RTAA* for real-time motion planning, and a Control Barrier Function safety filter — into a cohesive pipeline for safe waypoint navigation in unknown environments. The central design principle throughout is separation of concerns: each component has a well-defined responsibility, and the components compose so that the failure modes of one are covered by another. RTAA* handles global navigation but provides no safety guarantee; the CBF provides the safety guarantee but cannot navigate around obstacles alone; the UKF provides state estimates sufficient for control and planning; and LiDAR provides the obstacle map that both the planner and safety filter operate on.

Three experimental findings are worth highlighting. First, the CBF safety filter is necessary but not sufficient: removing it leads to physical collisions when planning or estimation errors carry the drone across an obstacle boundary, while using it alone without a planner leads to deadlock at local minima. Second, the hierarchy between planning and safety radii ($r_{\text{plan}} < r^{\text{safe}}$) is critical: it ensures that the path found by RTAA* already satisfies the safety constraint in nominal conditions, so the CBF acts as a true backstop rather than fighting the planner. Third, the activation gating condition for the CBF — enforcing constraints only when the drone is both within the lookahead distance and approaching an obstacle — is essential to prevent the safety filter from interfering with navigation at range.

A. Limitations

The main limitation of the current system is that the UKF relies on GPS for absolute position, which limits deployability to outdoor or GPS-available environments. The LiDAR center estimation error of approximately $0.15\,\text{m}$ means the safety guarantee is only as strong as the radius inflation margin $\delta = 0.20\,\text{m}$: an obstacle whose center is estimated with error greater than δ could result in a true safety radius violation even when the CBF constraint reports satisfaction.

B. Future Work

The most natural extension is replacing GPS with LiDAR-based localization. Using scan matching or an occupancy grid SLAM algorithm [6] would remove the GPS dependency entirely and make the system deployable indoors. In this setting, the same LiDAR that currently provides only obstacle detection would simultaneously provide self-localization, enabling a fully GPS-free pipeline.

A second direction is deployment on real hardware. The Crazyflie 2.1 platform and the Crazyswarm2 ROS2 driver provide a natural target: the geometric controller, UKF, and CBF formulations used here are standard and transfer directly to real hardware, with the main engineering challenge being latency management across the ROS2 communication layer.

Finally, the CBF formulation could be extended to handle dynamic obstacles by incorporating velocity estimates of detected objects into the barrier function. The current formulation assumes static obstacles; a time-varying barrier function $h(p, t)$ that accounts for predicted obstacle motion would extend safety guarantees to environments with moving objects, at the cost of additional perception complexity.

References

[1] S. Liu, K. Mohta, S. Shen, and V. Kumar, "Search-based motion planning for quadrotors using linear quadratic minimum time control," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, pp. 2872–2879.

[2] J. Tordesillas, B. T. Lopez, and J. P. How, "FASTER: Fast and safe trajectory planner for flights in unknown environments," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2019, pp. 1934–1940.

[3] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, "Control barrier functions: Theory and applications," in *Proceedings of the European Control Conference (ECC)*, 2019, pp. 3420–3431.

[4] E. A. Wan and R. van der Merwe, "The unscented Kalman filter for nonlinear estimation," in *Proceedings of the IEEE Adaptive Systems for Signal Processing, Communications, and Control Symposium (ASSPCC)*, 2000, pp. 153–158.

[5] R. Mahony, T. Hamel, and J.-M. Pflimlin, "Nonlinear complementary filters on the special orthogonal group," *IEEE Transactions on Automatic Control*, vol. 53, no. 5, pp. 1203–1218, 2008.

[6] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA: MIT Press, 2005.

[7] S. Koenig and M. Likhachev, "Real-time adaptive A*," in *Proceedings of the International Joint Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2006, pp. 281–288.

[8] Q. Nguyen and K. Sreenath, "Exponential control barrier functions for enforcing high relative-degree safety-critical constraints," in *Proceedings of the American Control Conference (ACC)*, 2016, pp. 322–328.

[9] T. Lee, M. Leok, and N. H. McClamroch, "Geometric tracking control of a quadrotor UAV on SE(3)," in *Proceedings of the IEEE Conference on Decision and Control (CDC)*, 2010, pp. 5420–5425.

[10] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2012, pp. 5026–5033.

[11] B. Stellato, G. Banjac, P. Goulart, A. Bemporad, and S. Boyd, "OSQP: An operator splitting solver for quadratic programs," *Mathematical Programming Computation*, vol. 12, no. 4, pp. 637–672, 2020.